Architecture Design Template Slides

A connected engineering record from need to release

Darian Muresan, Ph.D.
Stevens Institute of Technology
October 2026

Template v2.33



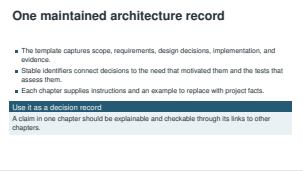
Present the deck as an overview of the DSN template, not as a completed project architecture. The manual remains the authoritative source.

One maintained architecture record

- The template captures scope, requirements, design decisions, implementation, and evidence.
- Stable identifiers connect decisions to the need that motivated them and the tests that assess them.
- Each chapter supplies instructions and an example to replace with project facts.

Use it as a decision record

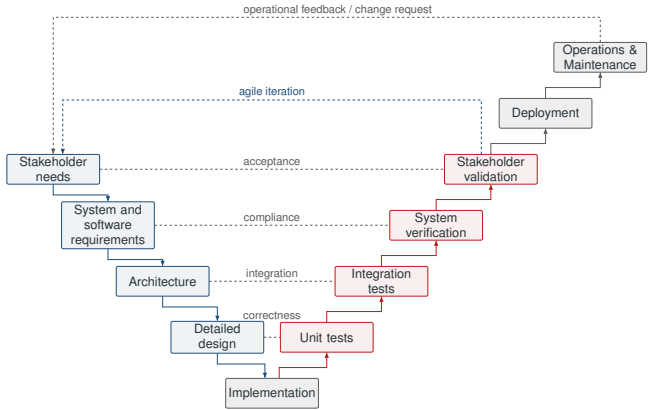
A claim in one chapter should be explainable and checkable through its links to other chapters.

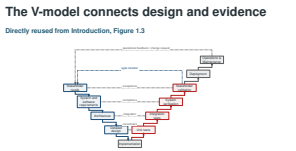


Distinguish the template from a completed design. Its examples are placeholders; project owners must supply and review actual decisions and evidence.

The V-model connects design and evidence

Directly reused from Introduction, Figure 1.3

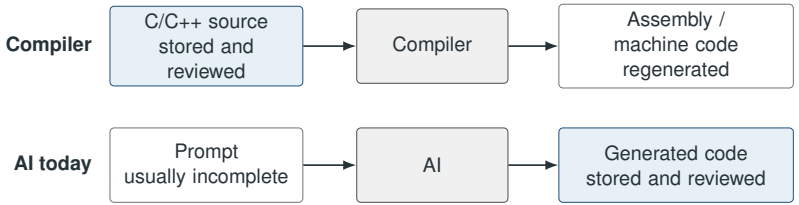




Walk down the left side from needs to implementation, then up the right side through tests and acceptance. Horizontal links state what each evidence level assesses. Point out agile iteration and operational feedback. The diagram source is shared with the manual.

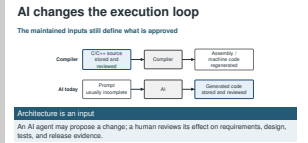
AI changes the execution loop

The maintained inputs still define what is approved



Architecture is an input

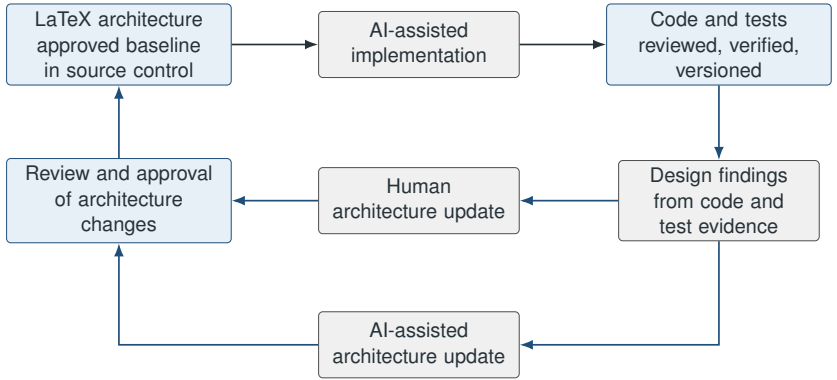
An AI agent may propose a change; a human reviews its effect on requirements, design, tests, and release evidence.



This is Figure 1.1 in the DSN introduction, adapted from MSE Figure 2.1. Explain that generated code is reviewed, not treated as the sole maintained design source.

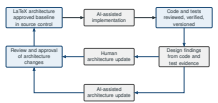
Architecture updates close the AI loop

Directly reused from Introduction, Figure 1.2



Architecture updates close the AI loop

Directly reused from Introduction, Figure 1.2



Describe the sequence: approved architecture and requirements enter the task; implementation and tests expose findings; a human approves changes to the baseline before another task uses it. The source figure belongs to the Introduction chapter.

Linked evidence makes architecture navigable

Link to evidence wherever possible

Use stable identifiers and hyperlinks so readers can follow a design claim to its rationale, implementation, and supporting results.

- **Navigate directly:** link requirements, use cases, figures, tables, tests, and evidence to their authoritative definitions.
- **Trace in both directions:** follow a need toward its implementation and evidence; trace a result back to the need it supports.
- **Maintain one source:** reference authoritative artifacts and keep mappings in the traceability register.

The architecture is a connected engineering record maintained like code.

Linked evidence makes architecture navigable
Link to evidence wherever possible
Use stable identifiers and hyperlinks so readers can follow a design claim to its rationale, implementation, and supporting results.
- **Navigate directly:** link requirements, use cases, figures, tables, tests, and evidence to their authoritative definitions.
- **Trace in both directions:** follow a need toward its implementation and evidence; trace a result back to the need it supports.
- **Maintain one source:** reference authoritative artifacts and keep mappings in the traceability register.

The architecture is a connected engineering record maintained like code.

Sources: DSN Document Guide and Traceability chapters. Emphasize using linked evidence as much as possible to make navigation, impact analysis, and traceability easy. Stable IDs and hyperlinks connect the record; the canonical register records its mappings. Links help reviewers find evidence but do not establish that the evidence is sufficient or correct.

Think of the template as code for architecture

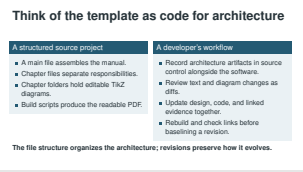
A structured source project

- A main file assembles the manual.
- Chapter files separate responsibilities.
- Chapter folders hold editable TikZ diagrams.
- Build scripts produce the readable PDF.

A developer's workflow

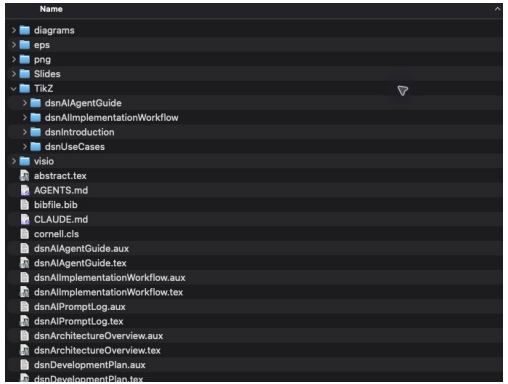
- Record architecture artifacts in source control alongside the software.
- Review text and diagram changes as diffs.
- Update design, code, and linked evidence together.
- Rebuild and check links before baselining a revision.

The file structure organizes the architecture; revisions preserve how it evolves.



The analogy is to a software source tree: the entry point includes responsibility-focused modules and shared assets, and a build produces a deliverable. Maintain the editable LaTeX and TikZ sources with the product revision. Record evidence versions and locations so links remain meaningful for that baseline. This is an authoring and configuration-management approach; the architecture itself is not executable software.

The template has a software-style source structure



Actual DSN `main/` folder; Finder screenshot (excerpt).

Entry point
`dsnManual.tex` assembles the chapters.

Modules
`dsnRequirements.tex`, `dsnLogicalView.tex`, etc. own chapter content.

Shared diagram sources
`TikZ/dsnIntroduction/`, etc. organize diagrams by chapter; slides reuse them.

Build and presentation
`makefileMacLinux` and `makefilePowerShell.ps1` build the manual; `Slides/` holds this deck.

The template has a software-style source structure



Actual DSN `main/` folder; Finder screenshot (excerpt).

Entry point
`dsnManual.tex` assembles the chapters.

Modules
`dsnRequirements.tex`, `dsnLogicalView.tex`, etc. own chapter content.

Shared diagram sources
`TikZ/dsnIntroduction/`, etc. organize diagrams by chapter; slides reuse them.

Build and presentation
`makefileMacLinux` and `makefilePowerShell.ps1` build the manual; `Slides/` holds this deck.

Screenshot captured from the actual ArchitectureDesignTemplate/main directory. It is cropped to the file list for legibility. This is an excerpt, not a complete directory inventory; the entry point and build scripts are elsewhere in the same directory. The .aux files visible in Finder are generated build intermediates. Editable chapter and TikZ sources remain authoritative, and the slide deck reuses the corresponding chapter diagrams.

The manual follows the engineering path

Define the system

Development plan
System context
User stories, requirements, use cases

Design the system

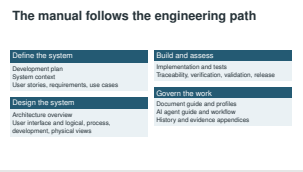
Architecture overview
User interface and logical, process, development, physical views

Build and assess

Implementation and tests
Traceability, verification, validation, release

Govern the work

Document guide and profiles
AI agent guide and workflow
History and evidence appendices



Use this map to locate detailed instructions in the DSN manual. Chapters are ordered to make dependencies visible; iteration can revisit earlier chapters.

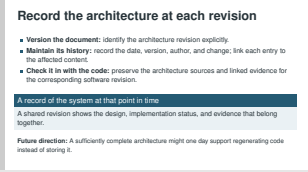
Record the architecture at each revision

- **Version the document:** identify the architecture revision explicitly.
- **Maintain its history:** record the date, version, author, and change; link each entry to the affected content.
- **Check it in with the code:** preserve the architecture sources and linked evidence for the corresponding software revision.

A record of the system at that point in time

A shared revision shows the design, implementation status, and evidence that belong together.

Future direction: A sufficiently complete architecture might one day support regenerating code instead of storing it.



Sources: DSN Document History, Implementation, Traceability, and Introduction chapters. The version identifies the document; the history explains its evolution; source control preserves the corresponding artifact set. Check in editable LaTeX and TikZ sources with the software and preserve evidence versions or stable references. The future direction is a possibility, not a current template capability: replacing stored code would require reproducible generation, controlled tools and dependencies, and verification of the generated implementation.

Organization makes the architecture usable

- **Give each chapter one responsibility:** state its purpose and expected result; keep definitions in their authoritative locations.
- **Keep AI instructions in the manual:** author them as `.tex` chapters so the compiled PDF provides contents, hyperlinks, and cross-references. Thin `.md` entry files point to that guidance.
- **Use UML wherever practical for software architecture:** choose diagrams that express structure, behavior, components, and deployment with consistent semantics.
- **Keep diagrams editable:** organize TikZ sources by chapter and link diagrams to the artifacts they describe.

Organization makes the architecture usable
- **Give each chapter one responsibility:** state its purpose and expected result; keep definitions in their authoritative locations.
- **Keep AI instructions in the manual:** author them as `.tex` chapters so the compiled PDF provides contents, hyperlinks, and cross-references. Thin `.md` entry files point to that guidance.
- **Use UML wherever practical for software architecture:** choose diagrams that express structure, behavior, components, and deployment with consistent semantics.
- **Keep diagrams editable:** organize TikZ sources by chapter and link diagrams to the artifacts they describe.

Sources: DSN Document Authoring Guide, AI Agent Usage Guide, and architecture-view chapters. Organization is essential for finding and maintaining authoritative information. AI skill definitions and execution instructions live in LaTeX and are published as navigable parts of the manual; AGENTS.md, CLAUDE.md, and SKILL.md remain thin discovery pointers. UML is the preferred notation for software architecture wherever it fits the question. TikZ is the editable drawing source, not a substitute for UML semantics.

Introduction

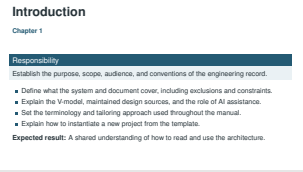
Chapter 1

Responsibility

Establish the purpose, scope, audience, and conventions of the engineering record.

- Define what the system and document cover, including exclusions and constraints.
- Explain the V-model, maintained design sources, and the role of AI assistance.
- Set the terminology and tailoring approach used throughout the manual.
- Explain how to instantiate a new project from the template.

Expected result: A shared understanding of how to read and use the architecture.



Source: Introduction chapter (dsnIntroduction.tex). Explain how this chapter contributes to the connected engineering record. Establish the purpose, scope, audience, and conventions of the engineering record. Point out the section Initializing a New Project from This Template.

Development Plan

Chapter 2

Responsibility

Define how the project is governed, planned, reviewed, and changed.

- Assign roles, milestones, review gates, and approval responsibilities.
- Record risks and the process for controlling changes.
- Decide domain-profile applicability with rationale, owner, and decision date.

Expected result: An actionable plan with visible ownership and decision gates.

Development Plan
Chapter 2
Responsibility
Define how the project is governed, planned, reviewed, and changed.
- Assign roles, milestones, review gates, and approval responsibilities.
- Record risks and the process for controlling changes.
- Decide domain-profile applicability with rationale, owner, and decision date.
Expected result: An actionable plan with visible ownership and decision gates.

Source: DevelopmentPlan chapter (dsnDevelopmentPlan.tex). Explain how this chapter contributes to the connected engineering record. Define how the project is governed, planned, reviewed, and changed.

System Context

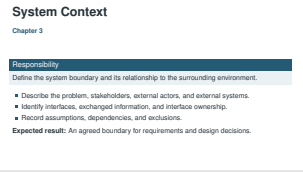
Chapter 3

Responsibility

Define the system boundary and its relationship to the surrounding environment.

- Describe the problem, stakeholders, external actors, and external systems.
- Identify interfaces, exchanged information, and interface ownership.
- Record assumptions, dependencies, and exclusions.

Expected result: An agreed boundary for requirements and design decisions.



Source: SystemContext chapter (dsnSystemContext.tex). Explain how this chapter contributes to the connected engineering record. Define the system boundary and its relationship to the surrounding environment.

User Stories

Chapter 4

Responsibility

Capture who needs a result and why that result has value.

- Identify the beneficiary, desired outcome, priority, and acceptance criteria.
- Give each story a stable US identifier.
- Link stakeholder intent to the requirements that realize it.

Expected result: A reviewable statement of stakeholder value and acceptance intent.

User Stories
Chapter 4
Responsibility
Capture who needs a result and why that result has value.
- Identify the beneficiary, desired outcome, priority, and acceptance criteria.
- Give each story a stable US identifier.
- Link stakeholder intent to the requirements that realize it.
Expected result: A reviewable statement of stakeholder value and acceptance intent.

Source: UserStories chapter (dsnUserStories.tex). Explain how this chapter contributes to the connected engineering record. Capture who needs a result and why that result has value.

Requirements

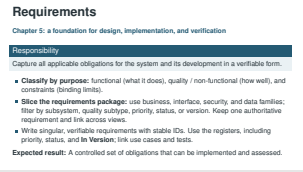
Chapter 5: a foundation for design, implementation, and verification

Responsibility

Capture all applicable obligations for the system and its development in a verifiable form.

- **Classify by purpose:** functional (what it does), quality / non-functional (how well), and constraints (binding limits).
- **Slice the requirements package:** use business, interface, security, and data families; filter by subsystem, quality subtype, priority, status, or version. Keep one authoritative requirement and link across views.
- Write singular, verifiable requirements with stable IDs. Use the registers, including priority, status, and **In Version**; link use cases and tests.

Expected result: A controlled set of obligations that can be implemented and assessed.



Source: Requirements chapter (dsnRequirements.tex), especially the identifier convention, functional and quality distinction, and category catalog. Families and review views are adaptable; each requirement has one primary register and a stable identifier. Specialized registers may include both functional and quality obligations. Filter or link rather than copying requirements. Leave In Version unassigned until implementation is complete in a controlled version.

The full scope of requirements

Chapter 5: one of the most important chapters

- **Include all types of requirements:** the chapter extends beyond the usual functional, quality, business, interface, security, and data requirements.
- **Developer requirements:** coding conventions, tools, build environments, testing practices, and development workflows.
- **AI requirements:** agent instructions, permitted actions, human review, and required evidence.
- **Naming requirements:** namespaces, class names, identifiers, files, and document conventions.
- **All applicable constraints:** technology, platforms, standards, regulation, resources, schedule, budget, and organizational policies.

Tailor to the domain: Use template tailoring profiles to customize requirements—for example, embedded systems may add timing, memory, power, and hardware constraints.

The full scope of requirements
Chapter 5: one of the most important chapters
- Include all types of requirements: the chapter extends beyond the usual functional, quality, business, interface, security, and data requirements.
- Developer requirements: coding conventions, tools, build environments, testing practices, and development workflows.
- AI requirements: agent instructions, permitted actions, human review, and required evidence.
- Naming requirements: namespaces, class names, identifiers, files, and document conventions.
- All applicable constraints: technology, platforms, standards, regulation, resources, schedule, budget, and organizational policies.

Tailor to the domain: Use template tailoring profiles to customize requirements—for example, embedded systems may add timing, memory, power, and hardware constraints.

Sources for tailoring: Introduction, Template Tailoring and Domain Profiles; Requirements, Domain Profile Impact on Requirements. Evaluate each applicable profile and turn relevant concerns into project requirements or links to existing requirements, with documented exclusions. Embedded timing, memory, power, and hardware constraints are illustrative concerns, not automatically applicable obligations. The breadth of this chapter is emphasized at the user's direction. Requirements may govern the product, its developers, AI agents, and the engineering process. These examples are concerns to capture, not new requirement identifier families. Reuse the template's defined families according to the primary obligation; follow its approval convention before introducing a family or scope code. Detailed AI, naming, and authoring guidance can remain in its authoritative chapter and be referenced from the requirements package without duplicating it.

Quality subtypes and scenarios

Chapter 5: prompts for design decisions

- **Use subtypes to prompt questions:** performance and capacity; reliability and recovery; security and privacy; usability and accessibility; maintainability and testability; portability and interoperability.
- **Ask what the system must handle:** peak load, failures, threats, different users, future changes, and external dependencies. Define measurable conditions and success criteria.
- **Optional extension: quality scenarios.** Describe the stimulus, operating conditions, affected part of the system, expected response, and measurable outcome.
- Reference scenarios from the **requirements package**; link them to the relevant requirement IDs, design decisions, and tests.

Example — recoverability: During acquisition, a connection fails; reconnect within 5 seconds and report any lost samples.

Quality subtypes and scenarios
Chapter 5: prompts for design decisions
Use subtypes to prompt questions: performance and capacity; reliability and recovery; security and privacy; usability and accessibility; maintainability and testability; portability and interoperability.
Ask what the system must handle: peak load, failures, threats, different users, future changes, and external dependencies. Define measurable conditions and success criteria.
Optional extension: quality scenarios. Describe the stimulus, operating conditions, affected part of the system, expected response, and measurable outcome.
Reference scenarios from the requirements package; link them to the relevant requirement IDs, design decisions, and tests.
Example — recoverability: During acquisition, a connection fails; reconnect within 5 seconds and report any lost samples.

Source for quality subtypes: the Requirements chapter category catalog. This list prompts consideration; select applicable concerns and justify omissions. Quality scenarios are an optional extension proposed for this presentation, not an existing scenario register in the template. Give each scenario a stable reference in the requirements package and link it to measurable quality requirements and verification evidence. The recovery example is illustrative, not a project requirement or an agreed threshold. A fuller scenario can identify the stimulus source, stimulus, environment, affected artifact, response, and response measure.

Use Cases

Chapter 6

Responsibility

Describe how actors interact with the system to achieve their goals.

- Show actors and goals inside a clear UML system boundary.
- Give each UC identifier its own standard use-case table.
- Record triggers, preconditions, main and alternate flows, outcomes, and links.

Expected result: Complete interaction scenarios tied to requirements and tests.

Use Cases
Chapter 6
Responsibility
Describe how actors interact with the system to achieve their goals.
- Show actors and goals inside a clear UML system boundary.
- Give each UC identifier its own standard use-case table.
- Record triggers, preconditions, main and alternate flows, outcomes, and links.
Expected result: Complete interaction scenarios tied to requirements and tests.

Source: UseCases chapter (dsnUseCases.tex). Explain how this chapter contributes to the connected engineering record. Describe how actors interact with the system to achieve their goals.

Architecture Overview

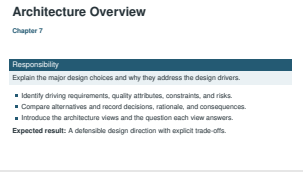
Chapter 7

Responsibility

Explain the major design choices and why they address the design drivers.

- Identify driving requirements, quality attributes, constraints, and risks.
- Compare alternatives and record decisions, rationale, and consequences.
- Introduce the architecture views and the question each view answers.

Expected result: A defensible design direction with explicit trade-offs.



Source: ArchitectureOverview chapter (dsnArchitectureOverview.tex). Explain how this chapter contributes to the connected engineering record. Explain the major design choices and why they address the design drivers.

User Interface Design

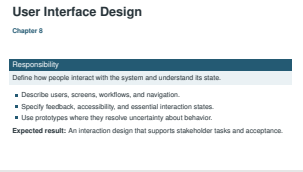
Chapter 8

Responsibility

Define how people interact with the system and understand its state.

- Describe users, screens, workflows, and navigation.
- Specify feedback, accessibility, and essential interaction states.
- Use prototypes where they resolve uncertainty about behavior.

Expected result: An interaction design that supports stakeholder tasks and acceptance.



Source: UserInterfaceDesign chapter (dsnUserInterfaceDesign.tex). Explain how this chapter contributes to the connected engineering record. Define how people interact with the system and understand its state.

Logical View

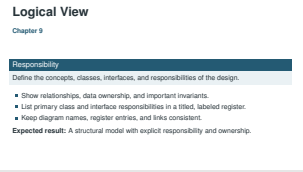
Chapter 9

Responsibility

Define the concepts, classes, interfaces, and responsibilities of the design.

- Show relationships, data ownership, and important invariants.
- List primary class and interface responsibilities in a titled, labeled register.
- Keep diagram names, register entries, and links consistent.

Expected result: A structural model with explicit responsibility and ownership.



Source: LogicalView chapter (dsnLogicalView.tex). Explain how this chapter contributes to the connected engineering record. Define the concepts, classes, interfaces, and responsibilities of the design.

Process View

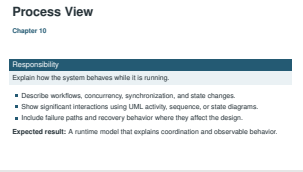
Chapter 10

Responsibility

Explain how the system behaves while it is running.

- Describe workflows, concurrency, synchronization, and state changes.
- Show significant interactions using UML activity, sequence, or state diagrams.
- Include failure paths and recovery behavior where they affect the design.

Expected result: A runtime model that explains coordination and observable behavior.



Source: ProcessView chapter (dsnProcessView.tex). Explain how this chapter contributes to the connected engineering record. Explain how the system behaves while it is running.

Development View

Chapter 11

Responsibility

Map the design to source organization and build artifacts.

- Identify modules, services, libraries, and their implementation boundaries.
- Define allowed dependencies and the rules that preserve those boundaries.
- Explain how source units form the deliverable artifacts.

Expected result: A code organization that developers can implement consistently.



Source: DevelopmentView chapter (dsnDevelopmentView.tex). Explain how this chapter contributes to the connected engineering record. Map the design to source organization and build artifacts.

Physical View

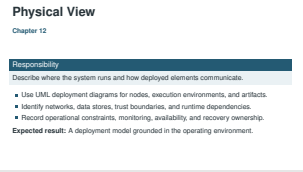
Chapter 12

Responsibility

Describe where the system runs and how deployed elements communicate.

- Use UML deployment diagrams for nodes, execution environments, and artifacts.
- Identify networks, data stores, trust boundaries, and runtime dependencies.
- Record operational constraints, monitoring, availability, and recovery ownership.

Expected result: A deployment model grounded in the operating environment.



Source: PhysicalView chapter (dsnPhysicalView.tex). Explain how this chapter contributes to the connected engineering record. Describe where the system runs and how deployed elements communicate.

Implementation

Chapter 13

Responsibility

Make the architecture reproducible through concrete implementation guidance.

- Record the technology baseline and required configuration.
- Provide the build and run commands.
- Define coding conventions and completion criteria that enforce the design.

Expected result: A repeatable path from design to a working implementation.

Implementation
Chapter 13
Responsibility
Make the architecture reproducible through concrete implementation guidance.
- Record the technology baseline and required configuration.
- Provide the build and run commands.
- Define coding conventions and completion criteria that enforce the design.
Expected result: A repeatable path from design to a working implementation.

Source: Implementation chapter (dsnImplementation.tex). Explain how this chapter contributes to the connected engineering record. Make the architecture reproducible through concrete implementation guidance.

Tests

Chapter 14

Responsibility

Define repeatable procedures that produce objective evidence.

- Assign stable TC identifiers and map tests to requirements.
- Specify environments, inputs, entry conditions, steps, and expected results.
- Define how execution results and evidence are recorded.

Expected result: Executable test specifications with observable pass criteria.

Tests
Chapter 14
Responsibility
Define repeatable procedures that produce objective evidence.
- Assign stable TC identifiers and map tests to requirements.
- Specify environments, inputs, entry conditions, steps, and expected results.
- Define how execution results and evidence are recorded.
Expected result: Executable test specifications with observable pass criteria.

Source: Tests chapter (dsnTests.tex). Explain how this chapter contributes to the connected engineering record. Define repeatable procedures that produce objective evidence.

Traceability

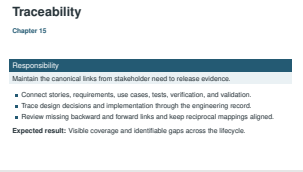
Chapter 15

Responsibility

Maintain the canonical links from stakeholder need to release evidence.

- Connect stories, requirements, use cases, tests, verification, and validation.
- Trace design decisions and implementation through the engineering record.
- Review missing backward and forward links and keep reciprocal mappings aligned.

Expected result: Visible coverage and identifiable gaps across the lifecycle.



Source: Traceability chapter (dsnTraceability.tex). Explain how this chapter contributes to the connected engineering record. Maintain the canonical links from stakeholder need to release evidence.

Verification

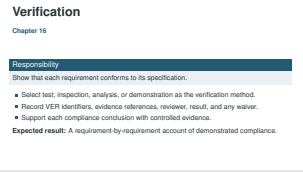
Chapter 16

Responsibility

Show that each requirement conforms to its specification.

- Select test, inspection, analysis, or demonstration as the verification method.
- Record VER identifiers, evidence references, reviewer, result, and any waiver.
- Support each compliance conclusion with controlled evidence.

Expected result: A requirement-by-requirement account of demonstrated compliance.



Source: Verification chapter (dsnVerification.tex). Explain how this chapter contributes to the connected engineering record. Show that each requirement conforms to its specification.

Validation

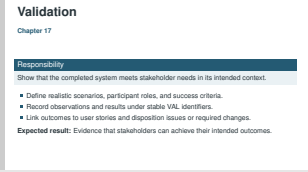
Chapter 17

Responsibility

Show that the completed system meets stakeholder needs in its intended context.

- Define realistic scenarios, participant roles, and success criteria.
- Record observations and results under stable VAL identifiers.
- Link outcomes to user stories and disposition issues or required changes.

Expected result: Evidence that stakeholders can achieve their intended outcomes.



Source: Validation chapter (dsnValidation.tex). Explain how this chapter contributes to the connected engineering record. Show that the completed system meets stakeholder needs in its intended context.

Release

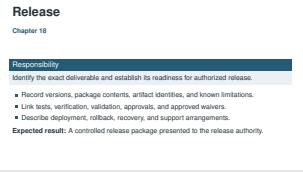
Chapter 18

Responsibility

Identify the exact deliverable and establish its readiness for authorized release.

- Record versions, package contents, artifact identities, and known limitations.
- Link tests, verification, validation, approvals, and approved waivers.
- Describe deployment, rollback, recovery, and support arrangements.

Expected result: A controlled release package presented to the release authority.



Source: Release chapter (dsnRelease.tex). Explain how this chapter contributes to the connected engineering record. Identify the exact deliverable and establish its readiness for authorized release.

Document Authoring Guide

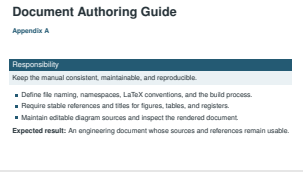
Appendix A

Responsibility

Keep the manual consistent, maintainable, and reproducible.

- Define file naming, namespaces, LaTeX conventions, and the build process.
- Require stable references and titles for figures, tables, and registers.
- Maintain editable diagram sources and inspect the rendered document.

Expected result: An engineering document whose sources and references remain usable.



Source: DocumentGuide chapter (dsnDocumentGuide.tex). Explain how this chapter contributes to the connected engineering record. Keep the manual consistent, maintainable, and reproducible.

Embedded Systems Tailoring Profile

Appendix B

Responsibility

Add the concerns needed when software interacts with constrained hardware.

- Record profile applicability in the Development Plan.
- Address hardware/software allocation, timing, resource budgets, and interfaces.
- Carry safety, security, toolchain, and hardware-test concerns into core chapters.

Expected result: The core architecture tailored to the embedded operating context.



Source: EmbeddedSystemsProfile chapter (dsnEmbeddedSystemsProfile.tex). Explain how this chapter contributes to the connected engineering record. Add the concerns needed when software interacts with constrained hardware.

AI Agent Usage Guide

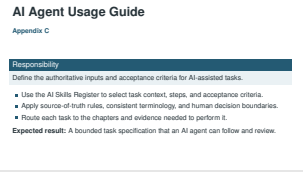
Appendix C

Responsibility

Define the authoritative inputs and acceptance criteria for AI-assisted tasks.

- Use the AI Skills Register to select task context, steps, and acceptance criteria.
- Apply source-of-truth rules, consistent terminology, and human decision boundaries.
- Route each task to the chapters and evidence needed to perform it.

Expected result: A bounded task specification that an AI agent can follow and review.


AI Agent Usage Guide
Appendix C
Responsibility
Define the authoritative inputs and acceptance criteria for AI-assisted tasks.
- Use the AI Skills Register to select task context, steps, and acceptance criteria.
- Apply source-of-truth rules, consistent terminology, and human decision boundaries.
- Route each task to the chapters and evidence needed to perform it.
Expected result: A bounded task specification that an AI agent can follow and review.

Source: AIAgentGuide chapter (dsnAIAgentGuide.tex). Explain how this chapter contributes to the connected engineering record. Define the authoritative inputs and acceptance criteria for AI-assisted tasks.

AI-Assisted Execution Workflow

Appendix D

Responsibility

Organize AI-assisted work from planning through integration and human review.

- Plan, assign, execute, integrate, check, and review the selected AI Skills.
- Choose one or more agents only when tools, authorization, and task structure permit.
- Control file ownership, correction attempts, evidence, and the review handoff.

Expected result: Coordinated execution with explicit checks and approval boundaries.



Source: AIImplementationWorkflow chapter (dsnAIImplementationWorkflow.tex). Explain how this chapter contributes to the connected engineering record. Organize AI-assisted work from planning through integration and human review.

AI Prompt Log

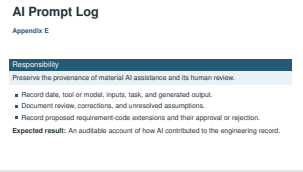
Appendix E

Responsibility

Preserve the provenance of material AI assistance and its human review.

- Record date, tool or model, inputs, task, and generated output.
- Document review, corrections, and unresolved assumptions.
- Record proposed requirement-code extensions and their approval or rejection.

Expected result: An auditable account of how AI contributed to the engineering record.



Source: AIPromptLog chapter (dsnAIPromptLog.tex). Explain how this chapter contributes to the connected engineering record. Preserve the provenance of material AI assistance and its human review.

Test Evidence

Appendix F

Responsibility

Retain detailed evidence supporting test and verification conclusions.

- Store or link reports, logs, screenshots, and other execution artifacts.
- Identify the test, result, environment, date, and product revision.
- Keep evidence readable, traceable, and appropriately redacted.

Expected result: Accessible evidence behind the results summarized in the main chapters.

Test Evidence
Appendix F
Responsibility
Retain detailed evidence supporting test and verification conclusions.
- Store or link reports, logs, screenshots, and other execution artifacts.
- Identify the test, result, environment, date, and product revision.
- Keep evidence readable, traceable, and appropriately redacted.
Expected result: Accessible evidence behind the results summarized in the main chapters.

Source: TestEvidence chapter (dsnTestEvidence.tex). Explain how this chapter contributes to the connected engineering record. Retain detailed evidence supporting test and verification conclusions.

Document History

Appendix G

Responsibility

Preserve the approved evolution of the architecture document.

- Record the date, version, author, change, and rationale.
- Link each entry to the affected chapter, section, table, figure, or artifact.
- Keep approved entries and place the newest approved change first.

Expected result: A durable record of what changed, why, and where.

Document History
Appendix G
Responsibility
Preserve the approved evolution of the architecture document.
▪ Record the date, version, author, change, and rationale.
▪ Link each entry to the affected chapter, section, table, figure, or artifact.
▪ Keep approved entries and place the newest approved change first.
Expected result: A durable record of what changed, why, and where.

Source: DocumentHistory chapter (dsnDocumentHistory.tex). Explain how this chapter contributes to the connected engineering record. Preserve the approved evolution of the architecture document.

How to use the template

Customize the architecture for a specific project

Template v2.33

How to use the template
Customize the architecture for a specific project
Template v2.33

Transition from the chapter responsibilities to applying the template. The following slides describe initialization, project boundaries, requirements, design, evidence, and tailoring.

Initialize a project with an AI agent

Manual Section 1.9: Initializing a New Project from This Template

Open the project copy in your AI tool, attach the available project evidence, and replace `[Project Name]`. Enter:

```
Read dsnAIAgentGuide.tex and use the AIS-02 task steps for [Project Name].
Execute using Appendix D. Review the supplied project evidence and create a gap
list before editing.  Identify missing scope, stakeholders, interfaces,
constraints, quality attributes, risks, evidence, and approvals.  Identify the
available domain profiles and ask me to confirm whether each is Applicable, Not
Applicable, or TBD. Read each Applicable or relevant TBD profile and add its
concerns to the gap list.  After the gap list is reviewed, replace only
supported placeholders and develop the document in dependency order.  Compile
the complete PDF and report unresolved human decisions.
```

Be ready to supply: the project namespace, metadata, profile decisions, and missing evidence.

Initialize a project with an AI agent
Manual Section 1.9: Initializing a New Project from This Template
Open the project copy in your AI tool, attach the available project evidence, and replace [Project Name]. Enter:
Read dsnAIAgentGuide.tex and use the AIS-02 task steps for [Project Name]. Execute using Appendix D. Review the supplied project evidence and create a gap list before editing. Identify missing scope, stakeholders, interfaces, constraints, quality attributes, risks, evidence, and approvals. Identify the available domain profiles and ask me to confirm whether each is Applicable, Not Applicable, or TBD. Read each Applicable or relevant TBD profile and add its concerns to the gap list. After the gap list is reviewed, replace only supported placeholders and develop the document in dependency order. Compile the complete PDF and report unresolved human decisions.
Be ready to supply: the project namespace, metadata, profile decisions, and missing evidence.

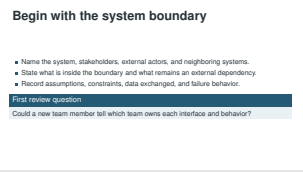
Sources: Introduction Section 1.9 (Section:InitializeNewProject) and AI Agent Guide, Initialize the Architecture Book (Section:AIInitializePrompt). The displayed request reproduces the manual's initialization prompt. Start with a project copy of the template and provide the AI tool access to that workspace and supplied evidence. Appendix D is the AI Implementation Workflow. AIS-02 requires a human-confirmed namespace before creating or renaming files and recorded profile applicability decisions. Review the gap list before detailed work; unsupported facts remain gaps.

Begin with the system boundary

- Name the system, stakeholders, external actors, and neighboring systems.
- State what is inside the boundary and what remains an external dependency.
- Record assumptions, constraints, data exchanged, and failure behavior.

First review question

Could a new team member tell which team owns each interface and behavior?



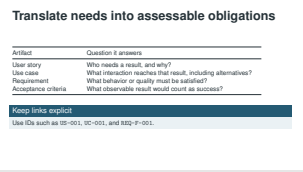
Point the audience to System Context before discussing internals. The boundary limits both requirements and architecture claims.

Translate needs into assessable obligations

Artifact	Question it answers
User story	Who needs a result, and why?
Use case	What interaction reaches that result, including alternatives?
Requirement	What behavior or quality must be satisfied?
Acceptance criteria	What observable result would count as success?

Keep links explicit

Use IDs such as `US-001`, `UC-001`, and `REQ-F-001`.



Explain that prose alone is insufficient. Requirements need implementation versions, status, and references in the template registers. Use cases use the template's standard use-case table.

Record decisions and their consequences

- Identify a decision as `ARC-001`.
- State its drivers: quality attributes, constraints, and evidence.
- Record the selected option, rationale, alternatives, and trade-offs.
- Update the decision when findings change the architecture.

Example

An asynchronous queue improves resilience to downstream outages, but introduces eventual consistency.

Record decisions and their consequences
Identify a decision as ARC-001.
State its drivers: quality attributes, constraints, and evidence.
Record the selected option, rationale, alternatives, and trade-offs.
Update the decision when findings change the architecture.
Example
An asynchronous queue improves resilience to downstream outages, but introduces eventual consistency.

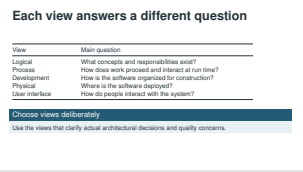
Use the example from the Architecture Overview chapter. The key is making both benefit and cost reviewable.

Each view answers a different question

View	Main question
Logical	What concepts and responsibilities exist?
Process	How does work proceed and interact at run time?
Development	How is the software organized for construction?
Physical	Where is the software deployed?
User interface	How do people interact with the system?

Choose views deliberately

Use the views that clarify actual architectural decisions and quality concerns.



The DSN overview says to use 4+1 views only when they help answer different architectural questions. Cross-reference the views when one element appears in several of them.

Make responsibilities explicit

- Give each class and interface in a class diagram a primary responsibility.
- Put those responsibilities in a titled, labeled register.
- Link diagram names and prose to the corresponding register rows.
- Keep the diagram, relationships, and register synchronized.

Review question

If an element changes, can its responsibility and affected interfaces be found immediately?

Make responsibilities explicit
Give each class and interface in a class diagram a primary responsibility.
Put those responsibilities in a titled, labeled register.
Link diagram names and prose to the corresponding register rows.
Keep the diagram, relationships, and register synchronized.
Review question
If an element changes, can its responsibility and affected interfaces be found immediately?

This reflects DOC-CLASS-001 and DOC-LINK-001 in the Logical View chapter. Mention that the register should be cited from the view text.

Trace a need through the full record

Need	Requirement	Use case	Test	Verification	Validation
US-001	REQ-F-001	UC-001	TC-001	VER-001	VAL-001

- Forward check: every obligation reaches an implementation and test.
- Backward check: every feature and test serves an approved need.
- Keep reciprocal links in the source chapters and the traceability register.

Trace a need through the full record

Need Requirement Use case Test Verification Validation
US-001 REQ-F-001 UC-001 TC-001 VER-001 VAL-001

- Forward check: every obligation reaches an implementation and test.
- Backward check: every feature and test serves an approved need.
- Keep reciprocal links in the source chapters and the traceability register.

The DSN Traceability chapter includes the canonical register. Use its example row to show how the IDs work, then discuss missing forward and backward links.

Verification and validation ask different questions

Verification	Validation
Did the implementation satisfy the specified requirement?	Does the delivered system meet the stakeholder need?
Evidence: test results, inspections, analyses, and linked `VER-001` records.	Evidence: acceptance results, operational scenarios, and linked `VAL-001` records.

Release cites the resulting evidence and unresolved risks.

Verification and validation ask different questions
Verification
Did the implementation satisfy the specified requirement?
Evidence: test results, inspections, analyses, and linked VER-001 records.
Validation
Does the delivered system meet the stakeholder need?
Evidence: acceptance results, operational scenarios, and linked VAL-001 records.
Release cites the resulting evidence and unresolved risks.

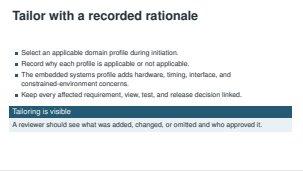
Use the V-model's horizontal relationships to explain which level of evidence is relevant. Do not equate passing tests with stakeholder acceptance.

Tailor with a recorded rationale

- Select an applicable domain profile during initiation.
- Record why each profile is applicable or not applicable.
- The embedded systems profile adds hardware, timing, interface, and constrained-environment concerns.
- Keep every affected requirement, view, test, and release decision linked.

Tailoring is visible

A reviewer should see what was added, changed, or omitted and who approved it.



Point to the Introduction's Domain Profiles section and the Embedded Systems Profile appendix. The base template remains the common structure.

Keep design and evidence connected

Template v2.33

Close by returning to the main idea: the architecture is a maintained engineering input. Questions can focus on how to tailor a view or evidence chain for a real project.